

# Optimizations and Changes Report

By Samarth Gupta

## Training/Testing Framework

Since my laptop lacks an NVIDIA GPU and using a GPU instance (e.g. Runpod) wasn't permitted, I decided to adopt a tier-testing mode approach. I built a *validate* mode that trains and validates on only 25,000 titles, keeping the same 90/10 train/validation split. A full *task train* run takes 2–3+ hours, whereas this reduced run finishes in 35–45 minutes, making iteration feasible. Throughout this report, I refer to these two configurations as **validate** (the 25,000-title test run) and **full** (the complete training run).

**On the 26th of June, between 1am - 2am** in the morning, the Hacker News Posts dataset experienced a version update which brought about a distributional shift, causing the validation losses to increase across both these modes. To ensure a fair comparison in this report, I've included the baseline validation losses below for *validate* and *full* modes, both pre-refresh and post refresh of the data. I will mainly run comparisons against the *validate* mode baseline, however I've attached a training loss curve below to compare my optimizations effect during the full run vs the baseline.

### Pre-Dataset Refresh Baseline

*Validate Mode:* **1.590**

*Full Mode:* **1.754**

### Post Dataset Refresh Baseline

*Validate Mode:* **1.993**

*Full Mode:* **1.872**

## Optimizations

### Optimizer: SGD → AdamW with Decoupled Weight Decay and LR Scheduling

The baseline optimizer was SGD, which applies a single global learning rate to every parameter. This is poorly suited to transformer training: different layers have very different gradient magnitudes and update frequencies, so a learning rate that works for

one layer is too large or too small for another. AdamW addresses this by maintaining a running mean and variance of each parameter's gradient history, scaling updates per-parameter so that infrequent or noisy gradients are handled appropriately (Karpathy, 2023). The decoupled weight decay (0.1) penalises large weights directly rather than through the gradient which is the key correction that separates AdamW from plain Adam, and which prevents weight decay from interfering with the adaptive gradient estimate.

A warmup and cosine decay schedule was introduced alongside the optimizer change. AdamW's internal moment estimates initialise at zero, meaning early gradient estimates are unreliable. Starting at full learning rate before those estimates have stabilised produces erratic early updates. A linear warmup over the first 5% of training steps gradually raises the learning rate as the estimates become more reliable. Cosine annealing then reduces it smoothly to  $lr \times 0.05$ , allowing fine-grained convergence without the abrupt cutoff of step decay. A floor of  $lr \times 0.05$  rather than zero was retained to preserve late-stage plasticity. Muon was considered as well, but AdamW was retained: its per-parameter adaptivity and stability was sufficient here.

**Result:** Validation loss **1.5899**  $\rightarrow$  **1.2847** ( $-0.3052$ ). This was the single largest improvement of the experiment, visible in Figure 1 as an immediate steep descent in the first two epochs that the baseline SGD run never achieves.

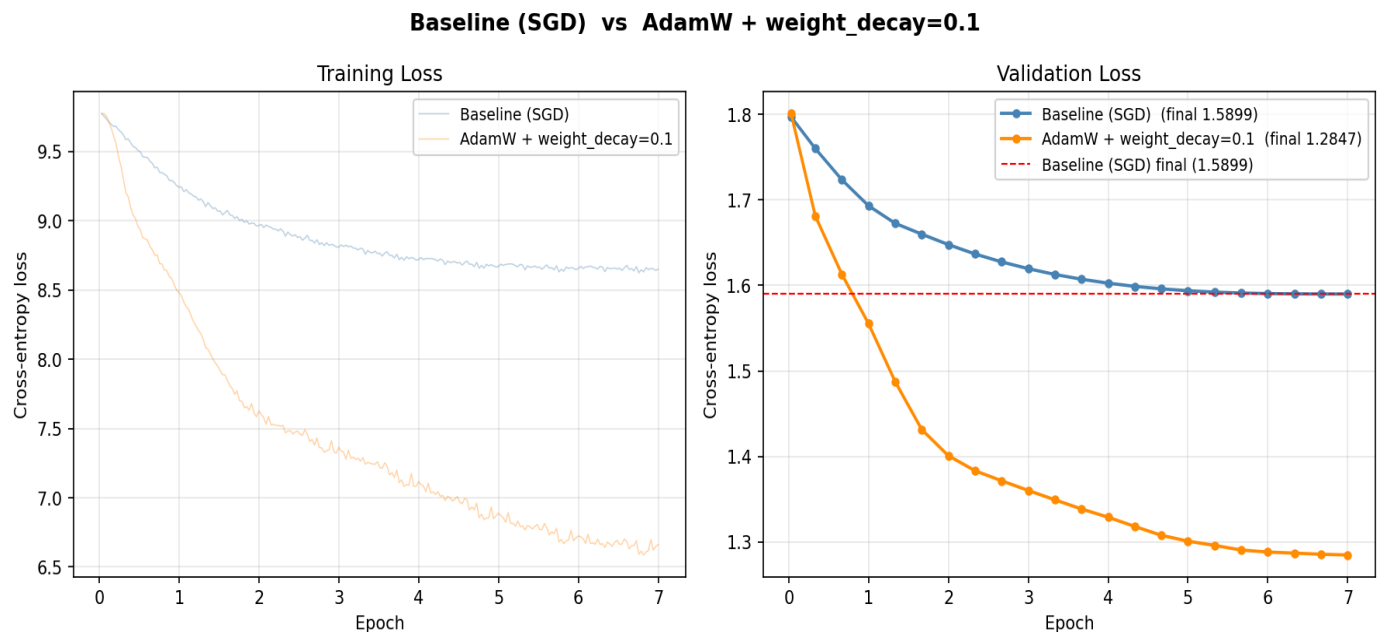


Figure 1

## Architectural Changes

### **Scaled Residual Initialisation — 1.2847 → 1.2547**

The output projections of each attention block and the down-projections of each MLP block were re-initialised with  $\text{std} = 0.02 / \sqrt{2 \times n\_layer}$ . These layers feed directly into the residual stream. With standard initialisation, variance accumulates across layers as each block adds its contribution to the residual stream. Scaling the projection weights by  $1 / \sqrt{2 \times n\_layer}$  keeps residual stream variance approximately constant at initialisation regardless of depth, giving AdamW a more stable starting point to build representations from (Karpathy, 2023).

### **RoPE + Parameter Groups — 1.2547 → 1.2360**

Learned absolute positional embeddings were replaced with Rotary Positional Encoding (RoPE). Absolute embeddings require the model to learn positional relationships entirely from data; RoPE directly encodes relative position by rotating query and key vectors in pairs, so the attention dot product between any two positions depends only on their offset. This inductive bias is well-suited to natural language where relative order matters more than absolute position. Alongside this, AdamW's parameter groups were split so that 1D parameters (biases, LayerNorm weights) received no weight decay hence regularising these is theoretically unsound as they do not contribute to overparameterisation in the same way weight matrices do.

### **Model Scaling (n\_layer 6→10, d\_model 512→640) — 1.2360 → 1.2269**

The baseline model was deliberately undersized. Depth and width were increased to  $n\_layer=10$  and  $d\_model=640$  (MLP hidden dim scaling proportionally to  $4 \times d\_model$ ). The head count was kept at 8, giving a head dimension of 80. Increasing  $n\_head$  was tested but reverted because narrower heads hurt per-head representational capacity more than the additional heads helped. A  $d\_model$  of 768 (matching GPT-2 small exactly), (Alammar, 2019) was also trialled, but  $n\_layer=10$  with  $d\_model=640$  outperformed it, suggesting the depth-to-width ratio mattered more than hitting a canonical size.

### **block\_size 128→256, batch\_size 64→32 — 1.2269 → 1.2227**

A longer context window exposes the model to more co-occurring headlines within each sequence, providing richer signal per step. Batch size was halved to offset the memory increase from doubling the context, with the mild side effect of noisier gradient estimates which are consistent with findings in large-batch training literature that slightly smaller batches can improve generalisation.

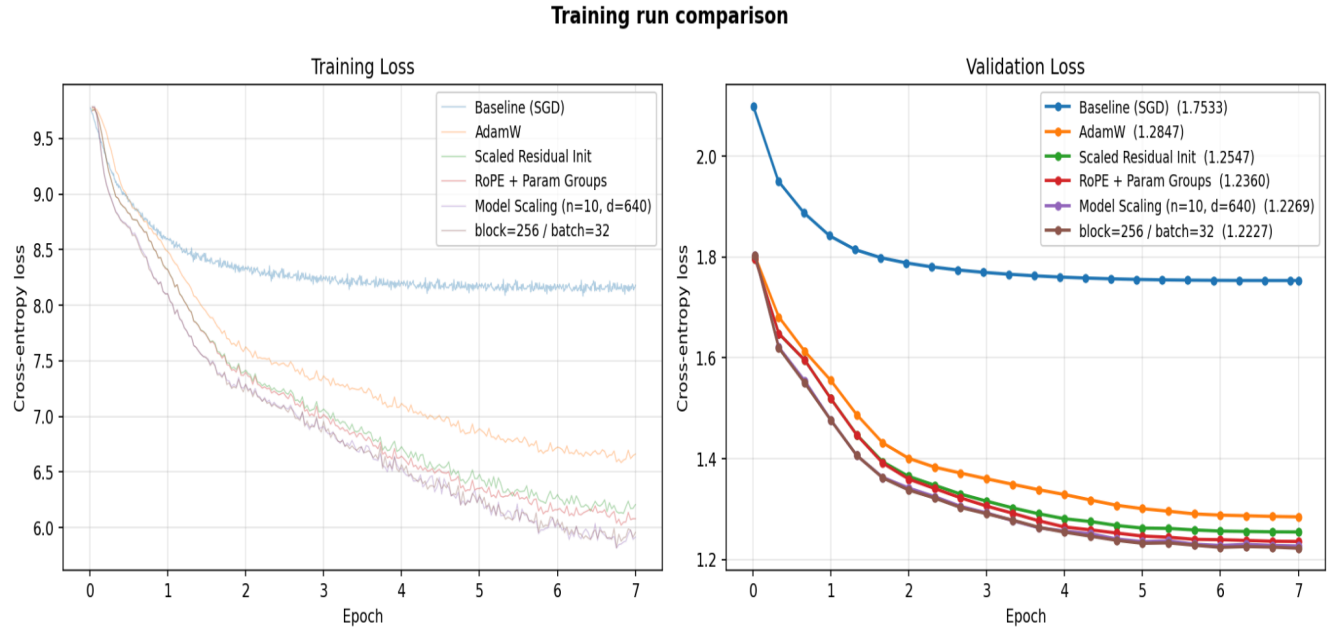


Figure 2

## Dataset Refresh

Because the BPE tokenizer is retrained on each dataset, the refresh also shifted the token vocabulary distribution: sequences that previously tokenized into familiar subword units now map to different token IDs, altering the co-occurrence statistics the model had learned. Hence the baseline for the *validate* mode shifted to 1.993 and the changes introduced to the architecture before the dataset refresh now sat at a validation loss of 1.433. The subsequent changes are hence evaluated against the newly established baselines.

## Speed Improvement & Parameter Changes

### Flash Attention + LR Warmup on New Data — 1.4328 → 1.4179

The manual attention computation was replaced with `F.scaled_dot_product_attention`, which dispatches to FlashAttention when available. FlashAttention rewrites the attention computation to tile in SRAM rather than materialising the full  $N \times N$  attention matrix in HBM, reducing memory bandwidth from  $O(N^2)$  to  $O(N)$  with no change to the mathematical result (Sai, 2025). This primarily increased throughput, enabling more stable training steps per epoch on the refreshed data.

### bf16 Precision

Training was switched from the default float32 to bfloat16 (brain float 16), which uses 8 exponent bits and 7 mantissa bits which is the same dynamic range as float32 but half the memory footprint. This is the key advantage over float16, whose 5 exponent bits make it prone to overflow and underflow during backpropagation, typically requiring loss scaling to compensate. bfloat16 avoids this, making it the preferred reduced-precision format for transformer training on modern hardware. This had no impact on the validation loss.

### Learning Rate Tuned to 6e-4 — 1.4179 → 1.4130

Post-refresh ablations over learning rates (4e-4, 5e-4, 6e-4) found 6e-4 to be the sweet spot with the cosine schedule. Too low and the model under-converged within 7 epochs; too high and late-epoch loss became noisy. n\_layer was also raised from 10 to 12 at this stage, contributing incrementally.

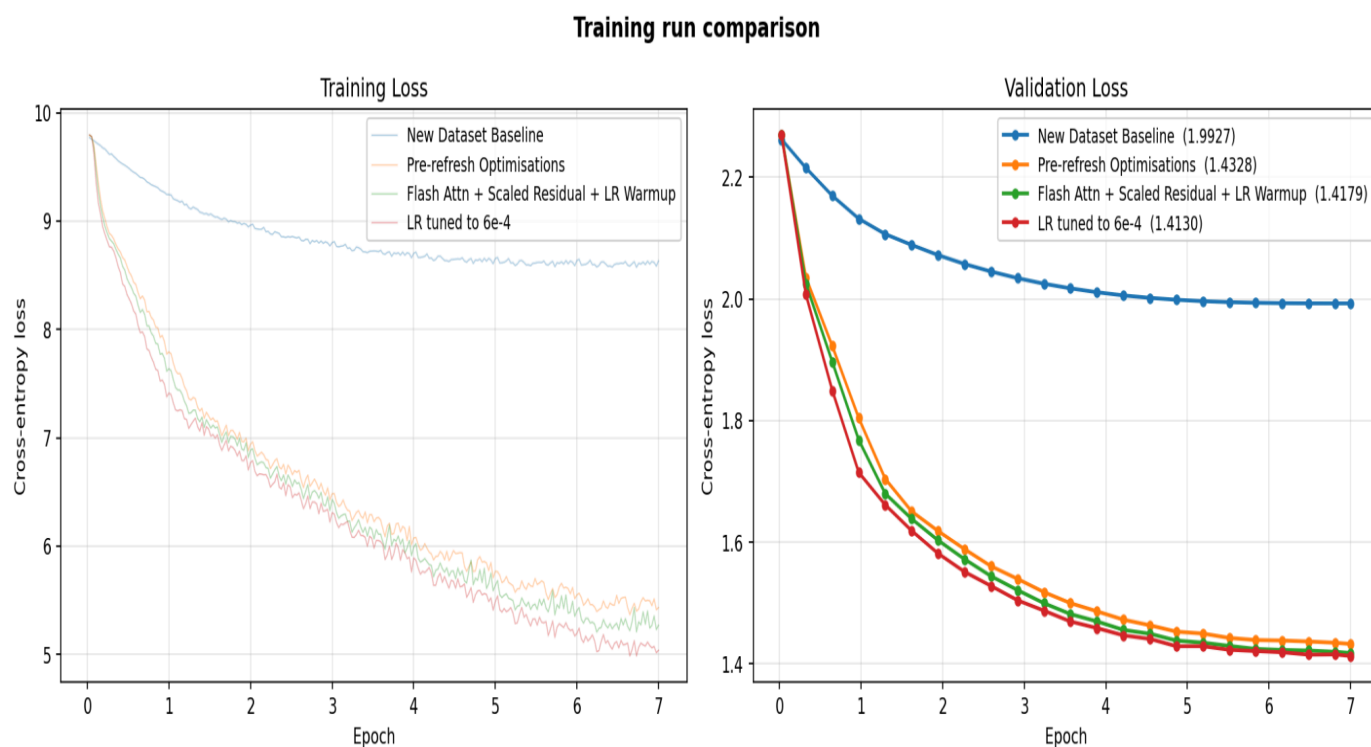


Figure 3

### Full Run: Baseline vs Final Model (Refreshed Dataset)

Figure 4 provides the headline comparison: both runs use the full 100,000-title dataset and the same refreshed data, making this the only direct apples-to-apples comparison in the report. The baseline config (SGD, n\_layer=6, d\_model=512) achieves a final validation loss of 1.8723. The

fully optimized model achieves **1.2143**, a reduction of 0.658, representing a **35% improvement** over baseline on the same data.

## Summary of Results

| Stage    | Change                                   | Validate Loss            |                          |
|----------|------------------------------------------|--------------------------|--------------------------|
|          |                                          | <i>Mode = validate</i>   | <i>Mode = full</i>       |
| Baseline | SGD, n_layer=6, d_model=512              | <i>Baseline = 1.5899</i> | <i>Baseline = 1.7540</i> |
| Fig 1    | AdamW + warmup / cosine schedule         | 1.2847                   | -                        |
| Fig 2    | Scaled residual initialisation           | 1.2547                   | -                        |
| Fig 2    | RoPE + parameter groups                  | 1.2360                   | -                        |
| Fig 2    | Model scaling (n_layer 10, d_model 640)  | 1.2269                   | -                        |
| Fig 2    | block_size=256, batch_size=32            | 1.2227                   | -                        |
| —        | <i>Dataset refresh (26 Jun 2026)</i>     | <i>Baseline = 1.9927</i> | <i>Baseline = 1.8723</i> |
| Fig 3    | Pre-refresh architecture on new data     | 1.4328                   | -                        |
| Fig 3    | Flash Attention + LR warmup              | 1.4179                   | -                        |
| Fig 3    | bfloat16                                 | 1.4179                   | -                        |
| Fig 3    | Learning rate tuned to 6e-4 (n_layer 12) | 1.4130                   | 1.2143                   |

**Full-run comparison (Figure 4, 100,000 titles, refreshed data):** baseline 1.8723 → final 1.2143, a reduction of 0.658 (**35% improvement**).

## Training run comparison

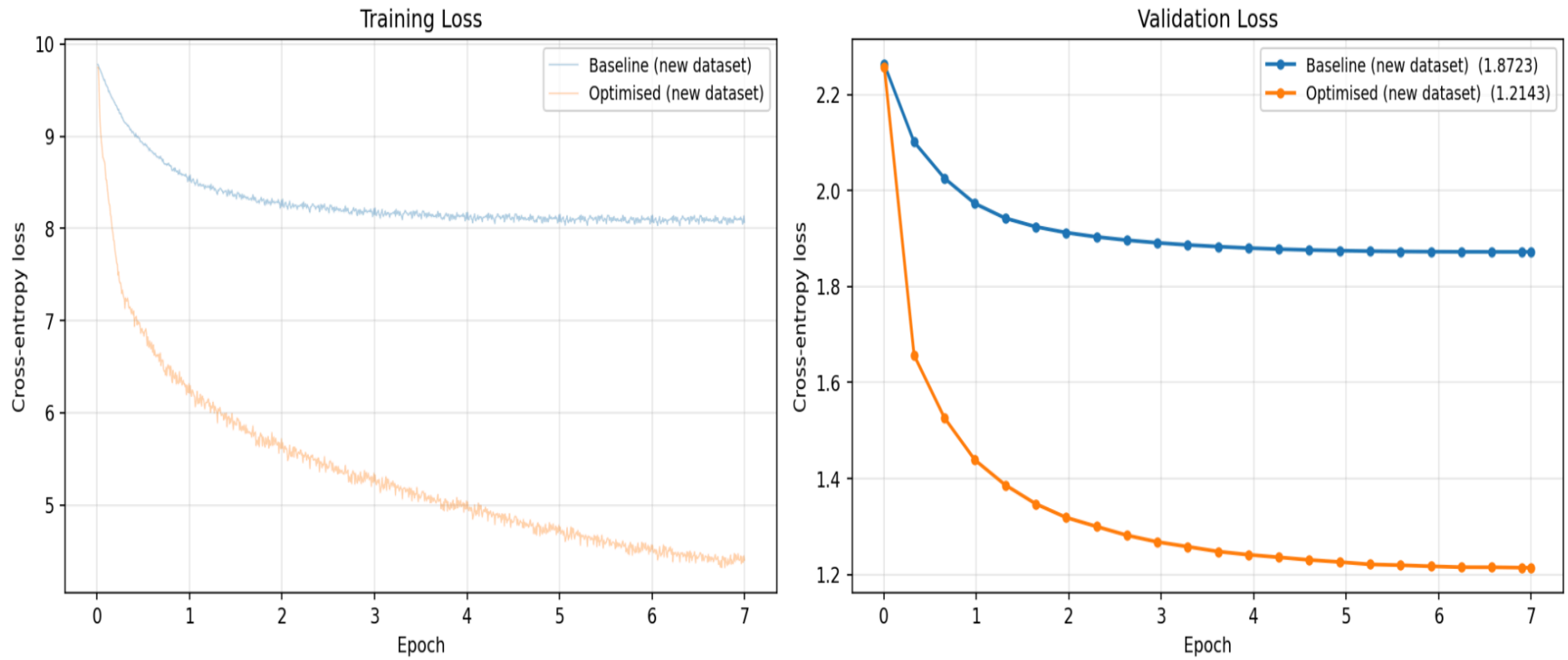


Figure 4

## Refactoring & Documentation

Prior to my refactoring, `train.py` was monolithic: hyperparameters, dataset loading, evaluation, logging, and the training loop all lived in a single file. This violates the Single Responsibility Principle (SRP), since one module ends up handling many distinct concerns, producing fragmented code with blurred boundaries between functions. It also makes the codebase harder to extend as more optimizations were layered onto the model, locating and modifying the relevant logic became increasingly error-prone.

I therefore refactored the codebase using Andrej Karpathy's nanoGPT (Karpathy, 2022) as a reference implementation, improving readability without compromising correctness. nanoGPT was adopted as a convention because it is a small-scale GPT implementation, mirroring the kind of model the Mainrun challenge targets.

| Module                 | Responsibility                                                              |
|------------------------|-----------------------------------------------------------------------------|
| <code>config.py</code> | <b>Hyperparameters</b> and <b>GPTConfig</b> dataclasses                     |
| <code>model.py</code>  | <b>CausalSelfAttention</b> , <b>MLP</b> , <b>Block</b> , <b>GPT</b> classes |
| <code>data.py</code>   | Dataset loading, BPE tokenizer, batch iterators                             |
| <code>utils.py</code>  | Logging, environment checks                                                 |
| <code>train.py</code>  | Training loop only (142 lines, down from 348)                               |

Documentation follows the same convention: concise inline comments above non-obvious logic rather than verbose docstrings. Class-level summaries describe what each component does, with levelled documentation consistent throughout nested functions inside classes.

## Thank you!!

This was a challenge I enjoyed! Having had to navigate through the issues regarding the GPU initially, to learning about the GPT-2, to experimenting what worked and didn't work in my iterations numerous times, I persisted, debugged and once again learnt something cool outside my Software Engineering scope that I had not encountered before. Thank you Maincode!



## References

Alammar, J. (2019, August 12). *The illustrated GPT-2 (Visualizing transformer language models)* [Blog post]. <https://jalammar.github.io/illustrated-gpt2/>

Karpathy, A. (2022). *nanoGPT* [Computer software]. GitHub. <https://github.com/karpathy/nanoGPT>

Karpathy, A. (2023, January 17). *Let's build GPT: From scratch, in code, spelled out* [Video]. YouTube. <https://www.youtube.com/watch?v=kCc8FmEb1nY>

Sai, M. (2025). *Notes on implementing GPT-2 from scratch*. Weights & Biases. <https://wandb.ai/training-transformers-vast/gpt2-sai/reports/Notes-on-Implementing-GPT-2-from-scratch---VmIldzoxMjE4Nzg4NA>